

Computer Science Bagrut Simulation – 5 Units

Subject: Java / C# Fundamentals, Arrays, and Object-Oriented Programming

Time: 90 Minutes

Grade: 11/12

Instructions:

- Answer all questions in the spaces provided.
- You may write your code in Java or C#.
- You can assume all standard libraries are imported (הנח כי כל המחלקות הדרושות מיובאות).

Question 1: Arrays & Algorithmic Thinking (מערכים ואלגוריתמיקה)

An array of integers is considered a "Balanced Array" (מערך מאוזן) if its length is an even number, and the sum of the elements in its first half is exactly equal to the sum of the elements in its second half.

For example:

- The array `[2, 4, 1, 5, 0, 2]` is a **מעריך מאוזן** because the sum of the first half ($2+4+1 = 7$) equals the sum of the second half ($5+0+2 = 7$).
- The array `[1, 2, 3, 4]` is **not** balanced ($1+2 \neq 3+4$).

Your Task:

Write a method (כתוב פעולה) named `isBalanced` that receives an array of integers `arr`. The method should return `true` if `arr` is a **מעריך מאוזן**, and `false` otherwise.

(Note: You must check if the array length is even. If it is odd, it cannot be balanced).

```
public static boolean isBalanced(int[] arr) {  
    int mid = arr.length / 2;  
    int sum1 = 0;  
    int sum2 = 0;  
  
    for (int i = 0; i < mid; i++) {  
        sum1 = sum1 + arr[i];  
    }  
  
    for (int i = mid; i < arr.length; i++) {  
        sum2 = sum2 + arr[i];  
    }  
}
```

```
return sum1 == sum2;
```

}

Question 2: Object-Oriented Programming (תכנות מונחה עצמים)

A local high school is organizing a hackathon and needs a system to manage the participating teams.

Part A (סעיף א):

Write the class `Team` (קבוצה). The class should include the following properties:

- `teamName` (String) - The name of the team.
- `numMembers` (int) - The number of students in the team.
- `needsComputers` (boolean) - `true` if the team needs the school to provide computers, `false` if they are bringing their own laptops.

1. Write the constructor method for the class.

2. Write standard `get` and `set` methods for all the attributes (פעולות מאחזרות וקובעות).

```
public class Team {  
    private String teamName;  
    private int numMembers;  
    private boolean needsComputers;  
  
    public Team(String teamName, int numMembers,  
                boolean needsComputers) {  
        this.teamName = teamName;  
        this.numMembers = numMembers;  
        this.needsComputers = needsComputers;  
    }  
}
```

```
public String getTeamName () {return teamName;}  
  
public void setTeamName(String Name) {  
    this.teamName = Name;  
}
```

```
public String getNumMembers () {return NumMembers;}  
  
public void setNumMembers(int Num ) {  
    this.NumMembers = Num ;  
}
```

```
public String getNeeds Computers () {return Needs Computers;}  
  
public void setNeeds Computers(boolean needs) {  
    this.Needs Computers = needs;  
}  
}
```

Part B (סעיף ב):

Write the class `Hackathon` (האקתון). The class manages an array of `Team` objects.
Properties:

- `teams` - An array of `Team` objects.
- `maxTeams` - The maximum number of teams allowed to register.
- `currentTeams` - The current number of teams registered.

1. Write a constructor that receives `maxTeams` and initializes the array accordingly. `currentTeams` should start at 0.
2. Write a method `public boolean addTeam(Team t)` (or `public bool AddTeam` in C#). The method should add the team `t` to the first available spot in the array if there is room. It should return `true` if the team was added successfully, and `false` if the hackathon is already full (אין מקום פנוי).
3. Write a method `public int countComputersNeeded()`. This method should calculate and return the total number of computers the school needs to provide. Assume that if a team needs computers (`needsComputers == true`), the school must provide one computer for *each member* of that team.

```

public class Hackathon {
    private Team[] teams;
    private int maxTeams;
    private int currentTeams;

    public Hackathon (int maxTeams) {
        this.maxTeams = maxTeams;
        this.teams = new Team[maxTeams];
        this.currentTeams = 0;
    }

    public boolean addTeam (Team t) {
        if (currentTeams < maxTeams) {
            teams[currentTeams] = t;
            currentTeams++;
            return true;
        }
        return false;
    }
}

```

```
public int countComputersNeeded() {
```

```
    int total = 0;
```

```
    for (int i = 0; i < currentTeams; i++) {
```

```
        if (teams[i].getNeedsComputers() == true) {
```

```
            total = total + teams[i].getNumMembers();
```

```
        }
```

```
    }
```

```
    return total;
```

```
}
```

```
}
```

Question 3: String Manipulation (מחרוזות)

A string is defined as a "Strong Password" (סיסמה חזקה) if it meets **all** of the following conditions:

1. Its length is at least 8 characters.
2. It contains at least one uppercase letter ('A' - 'Z').
3. It contains at least one digit ('0' - '9').

Your Task:

Write a static method named `isValidPassword` that receives a string `pass`. The method must check if the string meets all the conditions of a **סיסמה חזקה**. It should return `true` if it is a strong password, and `false` otherwise.

(Hint: You may use standard string methods like `charAt` and `length` in Java, or index access `[]` and `Length` in C#).

```
public static boolean isValidPassword(String pass) {  
    if (pass.length() < 8) { return false; }  
  
    boolean hasUpper = false;  
    boolean hasDigit = false;  
  
    for (int i = 0; i < pass.length(); i++) {  
        char c = pass.charAt(i);  
        if (c >= 'A' && c <= 'Z') {  
            hasUpper = true;  
        }  
        if (c >= '0' && c <= '9') {  
            hasDigit = true;  
        }  
    }  
}
```


}

return hasUpper & hasDigit;

}